

TableBite

Restaurant Ordering Platform

Ultimate Technical Planning Document

Version 2.0 — Merged Best-of MesaFlow + TableBite

Planning-only blueprint for a QR-first, multi-restaurant ordering platform built with Next.js, NestJS, PostgreSQL, Prisma, Socket.io, and Stripe.

Status: Pre-Implementation Review — Ready for Agent Execution

Date: August 2026

Scope Decision: Multi-restaurant marketplace with multi-branch support per restaurant. Core QR ordering ships first; commissions & advanced marketplace tooling arrive in later phases.

CONFIDENTIAL — For review and approval before implementation begins.

Table of Contents

- 1. Executive Summary
- 2. Competitive Research Summary
- 3. Full Feature List (v1 MVP vs v2)
- 4. User Roles & Permissions Matrix
- 5. End-to-End User Flows
- 6. System Architecture
- 7. Database Schema
- 8. API Design
- 9. QR Code Flow in Detail
- 10. Payment Flow in Detail
- 11. Folder / Project Structure
- 12. Non-Functional Requirements
- 13. Implementation Roadmap
- 14. Open Questions / Decisions (Resolved + Remaining)
- Appendix A — Sources & Research Links
- Appendix B — Agent Execution Instructions

1. Executive Summary

1.1 Product Overview

TableBite is a full-stack, QR-first restaurant ordering platform that lets guests scan a table (or restaurant) QR code, open a branded live digital menu in the browser (no app install), customize items with variants and add-ons, place an order, pay securely via Stripe or pay-at-counter, and track status in real time. Restaurant teams receive a unified operations console for menu management, table mapping, kitchen display, staff roles, and analytics. The platform is multi-restaurant from day one: each restaurant is an isolated tenant that can itself operate multiple branches.

1.2 Target Users

- **Diners / Guests** — Want zero-friction, contactless ordering and live status without downloading an app.
- **Restaurant Owners & Managers** — Need modern tools to own the customer relationship, avoid 15–30 % third-party commissions, and control menu, staff, and operations.
- **Kitchen & Wait Staff** — Need real-time order tickets, status updates, and simple workflows that reduce table friction.
- **Platform Super-Admins** — Onboard restaurants, monitor health, configure commissions, and view platform-wide analytics.

1.3 Core Value Proposition

Speed and clarity for the guest; operational ownership and higher margins for the restaurant. Every live order becomes a structured, auditable record that can be routed to the correct branch, table, or kitchen station and later analyzed for sales and menu performance. Restaurants keep direct ownership of menu, customer data, and payments.

1.4 Key Differentiators

- **No app required** — QR → browser menu → order → pay → track.
- **Real-time status** — Socket.io (with Redis adapter) pushes updates to guest and kitchen simultaneously.
- **Stripe sandbox → live key isolation** — Environment variables keep test and production completely separate.
- **Multi-restaurant + multi-branch** — Tenant isolation from the start; branches inherit or override menus and hours.
- **Secure table-scoped QR tokens** — Opaque signed tokens prevent table spoofing; token rotation on reprint.
- **Snapshot-based order items** — Historical receipts stay accurate even when the live menu changes.
- **Modern TypeScript stack** — Next.js App Router, NestJS, Prisma, PostgreSQL, Tailwind + shadcn/ui.

1.5 Scope Choice (Resolved)

Decision: Multi-restaurant marketplace with multi-branch support per restaurant for v1. A pure single-brand multi-branch model (as in the MesaFlow plan) is too limited for the long-term product vision; a pure marketplace without branch awareness would force unnecessary complexity on chains. Platform admin, commission configuration, and basic payout tracking ship in Phase 6; the core ordering loop ships earlier.

2. Competitive Research Summary

Research combined insights from MesaFlow (Toast, Square, GloriaFood, Odoo, UpMenu, Elitale Restro, ERPNext Restaurant) and TableBite (Square, Toast, Grubhub, URY OSS, FloCafe, Satisfecho). The must-haves across all products are consistent.

2.1 Feature Comparison (Selected Products)

Feature	Square	Toast	GloriaFood	Odoo	UpMenu	Satisfecho / URY
QR Menu	Yes	Yes	Yes	Yes	Yes	Yes
App-free ordering	Yes	Yes	Yes	Yes	Yes	Yes
Dine-in + Pickup	Yes	Yes	Yes	Yes	Yes	Yes
Real-time status	Partial	Partial	Yes	Yes	Yes	Yes
Kitchen Display	No	Yes (KDS)	Limited	Yes	Yes	Yes
Stripe / Card	Own	Own	Yes	Yes	Yes	Yes
Item variants / add-ons	Yes	Yes	Yes	Yes	Yes	Yes
Staff roles	Yes	Yes	Yes	Yes	Yes	Yes
Multi-location / branch	Yes	Yes	Yes	Yes	Yes	Yes
Multi-restaurant platform	No	No	No	No	Limited	Yes
Commission-free direct	Yes	Yes	Yes	Yes	Yes	Yes
Open-source option	No	No	No	Yes	No	Yes

Sources synthesized from product docs and open-source repositories listed in Appendix A.

2.2 Must-Haves Adopted for v1

- Branded mobile-friendly menu with categories, images, search/filter, dietary tags.
- Table- or location-specific QR mapping with secure opaque tokens.
- Item customization (variants + add-ons / modifier groups).
- Cart → checkout → card (Stripe) or pay-at-counter.
- Real-time order status to guest and staff (Socket.io).
- Kitchen / order queue view and basic kitchen display.
- Role-based staff access (owner, manager, kitchen, waiter).
- Menu CRUD, availability toggles, table management, hours, basic analytics.
- Customer accounts (email + Google OAuth), order history, favorites, reviews.
- Platform admin for restaurant onboarding and basic multi-tenant controls.

2.3 Deferred to v2 (with reasoning)

- Full delivery fleet / DSP integrations — address validation, courier workflows, SLA complexity.
- Loyalty points, promo codes, gift cards, referral flows — growth features after core loop is proven.
- Deep inventory / recipe depletion — adds stock model complexity beyond simple availability toggles.
- SMS / push notifications beyond in-app WebSocket — third-party cost and compliance.
- Advanced marketing automation, reservations, table splitting, multi-language i18n.
- Native mobile apps — web-first + PWA readiness is sufficient for v1.
- Automated commission calculation & scheduled payouts — manual tracking first.

3. Full Feature List (v1 MVP vs v2)

3.1 Customer-Facing

Area	v1 (MVP)	v2 (Future)
Menu & Discovery	QR menu, browse / search / filter, dietary tags, item detail with images & modifiers	Saved carts across devices, richer personalization, social sharing
Ordering	Cart, customization, dine-in (table from QR) + pickup, special instructions	Delivery + DSP, group ordering, table splitting
Checkout & Pay	Guest or account checkout, Stripe card, pay-at-counter / cash fallback, receipts	Split payments, tips, gift cards
Tracking	Real-time status stream (Placed → Confirmed → Preparing → Ready → Completed)	Map-based delivery tracking
Account	Email + Google OAuth, order history, reordering, favorites, saved addresses, ratings/reviews	Loyalty points, promo codes, referral flows

3.2 Restaurant Staff & Operations

Area	v1 (MVP)	v2 (Future)
Dashboard	Live order queue (Kanban or list), kitchen view, basic analytics	Advanced forecasting, multi-station load balancing, prep-time analytics
Menu	CRUD categories / items / variants / modifier groups, availability toggles, images, drag reorder	Scheduled menu changes (breakfast/lunch), versioning, inventory alerts
Tables & QR	Table CRUD, per-table QR generation & print, status (available/occupied)	Floor-plan visual editor, reservation integration
Staff	Role assignment (owner/manager/kitchen/waiter), branch scope	Scheduling, shift management
Fulfillment	Order status transitions, item-level ready, notes, browser print receipt	Dedicated KDS full-screen, ESC/POS printer, multi-station routing

3.3 Platform Admin

Area	v1 (MVP)	v2 (Future)
Onboarding	Restaurant registration, approve/reject, activate/deactivate	Document verification workflow, automated checks
Economics	Commission rate configuration (platform-wide or per restaurant), basic payout tracking	Automated calculation, scheduled payouts, disputes
Analytics	Platform-wide restaurants / orders / revenue overview	Churn, growth metrics, geographic heatmaps, fraud tooling
Support	Audit logs, basic restaurant listing management	Ticketing system, integration marketplace

4. User Roles & Permissions Matrix

Roles are either platform-scoped or restaurant/branch-scoped. A user can hold different roles across different restaurants. Every write action is guarded by role + tenant/branch scope. Reads respect membership and the guest's own data.

4.1 Role Definitions

- **Platform Super-Admin** — Full platform control: restaurants, commissions, payouts, global analytics, audit.
- **Platform Moderator** — Approve/reject restaurants, view platform analytics; cannot change commission rates or delete data.
- **Restaurant Owner** — Full control of their restaurant(s) and branches: menu, staff, tables, settings, refunds, analytics.
- **Restaurant Manager** — Menu, orders, tables, analytics, refunds; cannot manage staff roles or delete restaurant.
- **Kitchen Staff** — View kitchen queue, update item/order prep states, add prep notes; no customer PII beyond fulfillment needs.
- **Waiter** — Open/attach table orders, assist guests, update service steps, view live tables; cannot change menu pricing.
- **Registered Customer** — Own orders, favorites, addresses, reviews; cannot access any restaurant data.
- **Guest Customer** — Browse, cart, order, pay, track; no persistent profile until they create an account.

4.2 Permissions Matrix (Selected)

Permission	Super-Admin	Moderator	Owner	Manager	Kitchen	Waiter
View public menu	Y	Y	Y	Y	Y	Y
Place order (customer)	Y	Y	Y	Y	N	N
Update order status	Y	N	Y	Y	Y	Limited
Manage menu / items	N	N	Y	Y	R	R
Manage tables & QR	N	N	Y	Y	R	R
Manage staff roles	Y	N	Y	Limited	N	N
View analytics (rest.)	Y	Y	Y	Y	N	N
Process refunds	Y	N	Y	Y	N	N
Approve restaurants	Y	Y	N	N	N	N
Manage commissions	Y	N	N	N	N	N
View platform analytics	Y	Y	N	N	N	N
View customer PII	Y	R	Y	R	N	N

Y = Full, R = Read-only, N = None, Limited = scoped actions only.

5. End-to-End User Flows

5.1 Customer: Scan → Order → Pay → Track

1. Guest scans table QR → opaque signed URL resolves to restaurant + branch + table context.
2. Backend verifies signature, token version, active status, and branch open hours; issues short-lived guest session.
3. Live menu loads (categories, availability, images). Guest browses, searches, filters by dietary tags.
4. Guest opens item, selects variants/add-ons, sets quantity, adds special instructions, adds to cart.
5. Cart review → select order type (dine-in pre-filled from table, or switch to pickup).
6. Checkout: choose card (Stripe) or pay-at-counter. Backend calculates totals from server-side snapshot.
7. Card path: NestJS creates PaymentIntent / Checkout Session → frontend confirms → Stripe webhook confirms → order advances to placed/confirmed.
8. Cash path: order created as unpaid / pending-due; staff settles later.
9. Socket.io pushes status updates to guest: Confirmed → Preparing → Ready → Completed.
10. After completion guest can rate/review items and restaurant.

Mermaid-style (for diagramming):

```

flowchart TD
  A[Guest scans QR] --> B[Signed URL resolves restaurant+branch+table]
  B --> C[Menu loads with live availability]
  C --> D[Customize item + add to cart]
  D --> E[Select dine-in / pickup]
  E --> F[Checkout]
  F --> G{Payment}
  G -->|Card| H[Stripe PaymentIntent + webhook]
  G -->|Cash| I[Unpaid order created]
  H --> J[Order placed]
  I --> J
  J --> K[Real-time status to guest & kitchen]
```

5.2 Restaurant Onboarding & Daily Management

1. Owner registers → creates account → submits restaurant profile (name, address, hours, logo).
2. Status = Pending Approval. Platform admin reviews and approves → status = Active; owner notified.
3. Owner (or manager) creates categories, items, modifier groups, prices, images; toggles availability.
4. Defines tables / areas, generates per-table QR codes (printable), configures branch hours & timezone.
5. Invites staff by email and assigns roles with optional branch scope.
6. During service: live order board receives new tickets via WebSocket; staff move orders through states; out-of-stock items are toggled off instantly.
7. End-of-day: sales summary, payment mix, top items.

5.3 Kitchen Order Fulfillment

1. New order event → kitchen queue receives ticket (optionally grouped by station).
2. Kitchen staff starts prep, marks individual items ready (partial fulfillment supported).
3. When all items ready → order status auto-advances to Ready; guest notified.
4. Waiter marks served / completed. Notes and prep comments are preserved on the ticket.

5.4 Platform Admin Onboarding a Restaurant

1. Admin sees Pending list → reviews profile, owner, optional menu preview.
2. Approves → status Active, default commission & settings provisioned, owner receives access.
3. Admin can later adjust commission, deactivate, or view platform analytics.

6. System Architecture

Decoupled, TypeScript-first architecture. Frontend on Vercel (edge + SSR); backend + Postgres + Redis on a container-friendly host (Railway / Render / Fly.io). Real-time via Socket.io running inside the NestJS process with Redis adapter for horizontal scale.

6.1 Component Overview

- **Next.js (App Router)** — Customer menu, checkout, staff dashboards, platform admin. Tailwind + shadcn/ui + Radix. SSR for menu pages, client components for live status.
- **NestJS** — REST API, business rules, auth, order state machine, menu CRUD, analytics aggregation, Stripe webhooks, Socket.io gateway.
- **PostgreSQL + Prisma** — Source of truth. Prisma for type-safe queries, migrations, and seed data.
- **Socket.io + Redis** — Real-time order & kitchen events. Redis adapter enables multi-instance scaling; also used for rate-limit and short-lived session cache.
- **Stripe** — PaymentIntents / Checkout Sessions + webhooks. Test keys in development, live keys in production (separate env vars).
- **Object storage** — Cloudinary or S3 for menu images, logos, QR PNG exports (signed URLs).
- **QR service** — Server-side generation (qrcode package) of signed, versioned tokens; printable assets.
- **Email** — Transactional receipts and account actions (Resend / Postmark / SES).

6.2 High-Level Connection Diagram (textual)

```
[Customer / Staff Browser] ←HTTPS→ [Next.js on Vercel]
[Browser] ←WSS→ [Socket.io gateway inside NestJS]
[Next.js] ←REST JSON→ [NestJS API]
[NestJS] ←SQL→ [PostgreSQL]
[NestJS] ←Redis protocol→ [Redis (adapter + cache)]
[NestJS] ←HTTPS→ [Stripe API]
[Stripe] ←Webhook→ [NestJS /payments/stripe/webhook]
[NestJS] ←HTTPS→ [S3 / Cloudinary]
```

6.3 Why REST (not GraphQL) for v1

The domain is highly transactional, webhook-heavy, and role-driven. REST keeps order actions explicit, webhook payloads predictable, and NestJS guards simple. GraphQL can be added later for complex reporting or menu composition if it becomes a bottleneck.

6.4 Why Prisma

Excellent TypeScript integration, declarative migrations, easy seeding, and strong schema-as-code alignment with the monorepo shared types package. TypeORM remains a viable alternative if raw query flexibility is later preferred.

7. Database Schema

Core design principles (merged from both plans): (1) multi-tenant via `restaurant_id` / `organization_id`, (2) order and order-item snapshots so historical receipts remain accurate, (3) opaque QR tokens with version rotation, (4) normalized modifier groups reusable across items, (5) branch-scoped roles and hours.

7.1 Core Entities (abbreviated column list — full Prisma schema to be generated in Phase 1)

```

restaurant (id UUID PK, name, slug unique, status enum active/suspended/pending, logo_url,
commission_rate, created_at, updated_at)

branch (id UUID PK, restaurant_id FK, name, slug unique-per-restaurant, timezone, address, phone, status
active/inactive)

branch_hours (id, branch_id FK, day_of_week 0-6, open_time, close_time, is_closed bool)

user (id UUID PK, email citext unique, display_name, phone, is_email_verified, status active/blocked)

auth_identity (id, user_id FK, provider email/google/password, provider_account_id, password_hash,
refresh_token_hash)

role (id, code unique, name) -- platform_superadmin, platform_moderator, owner, manager, kitchen, waiter,
customer

user_role (user_id, role_id, restaurant_id nullable, branch_id nullable, PK composite)

menu_category (id, branch_id or restaurant_id, parent_id nullable, name, sort_order, is_visible)

menu_item (id, category_id FK, name, description, base_price numeric(12,2), currency char(3),
is_available, is_featured, dietary_tags text[])

menu_item_image (id, menu_item_id, url, alt_text, sort_order)

modifier_group (id, restaurant_id or branch_id, name, min_select, max_select, is_required)

modifier_option (id, modifier_group_id, name, price_delta, is_available)

menu_item_modifier_group (menu_item_id, modifier_group_id) -- M2M

restaurant_table (id, branch_id, label, area, capacity, status
available/occupied/reserved/out_of_service)

qr_code (id, branch_id, table_id nullable, token_hash unique, token_version int, is_active, expires_at)

order (id, branch_id, table_id nullable, customer_id nullable, order_type dine_in/pickup/delivery, status
placed/confirmed/preparing/ready/completed/cancelled, subtotal, tax, service_fee, discount, total,
currency, special_instructions, placed_at)

order_item (id, order_id, menu_item_id nullable, item_name_snapshot, unit_price_snapshot, quantity,
note, status queued/preparing/ready/served/cancelled)

order_item_modifier (id, order_item_id, modifier_option_id nullable, modifier_name_snapshot,
price_delta_snapshot)

payment (id, order_id, provider stripe/cash, provider_payment_id, amount, status
pending/authorized/captured/failed/refunded, currency, paid_at)

order_status_event (id, order_id, actor_user_id nullable, from_status, to_status, note, created_at)

saved_address, favorite, review, notification, audit_log -- as in MesaFlow schema

```

Rationale: Snapshots on `order_item` and `order_item_modifier` protect receipts. QR tokens are high-entropy hashes; the public URL never exposes raw table IDs. Modifier groups are normalized for reuse. `user_role` supports both restaurant-wide and branch-scoped assignments.

8. API Design

Domain-grouped REST endpoints. All write endpoints require appropriate role + tenant scope. Public menu endpoints are rate-limited and cacheable.

Auth

- POST /auth/signup — Create customer account (email/password)
- POST /auth/login — Issue JWT / session
- POST /auth/oauth/google — Start Google OAuth
- GET /auth/me — Current user + roles
- POST /auth/logout — Revoke session / refresh

Public Menu & QR

- GET /public/qr/:token — Resolve signed token → branch + table context + short-lived guest session
- GET /public/branches/:slug/menu — Live menu (categories, items, availability)
- GET /public/menu/search — Search / filter items
- GET /public/menu/item/:id — Item detail + modifiers

Orders

- POST /orders — Create draft order (from cart + context)
- GET /orders/:id — Fetch order (owner or staff scoped)
- GET /orders/:id/status — Current status + events
- POST /orders/:id/confirm — Confirm draft
- POST /orders/:id/cancel — Cancel (rules apply)
- PATCH /orders/:id/items/:itemId — Update note/qty before confirmation

Payments

- POST /payments/stripe/create-intent — Create PaymentIntent / Checkout Session
- POST /payments/stripe/webhook — Stripe webhook (signature verified)
- POST /payments/cash/mark-due — Mark pay-at-counter
- POST /payments/:id/refund — Issue refund (authorized roles)

Restaurants / Branches / Tables

- POST /restaurants — Register new restaurant (pending)
- GET /restaurants/:id — Profile
- POST /branches — Create branch
- PATCH /branches/:id — Update
- PATCH /branches/:id/hours — Hours
- POST /branches/:id/tables — Create tables
- POST /branches/:id/qr-codes — Generate / rotate QR codes

Menu Admin

- POST/PATCH/DELETE /admin/menu/categories — CRUD
- POST/PATCH/DELETE /admin/menu/items — CRUD + availability toggle
- POST/PATCH /admin/menu/modifier-groups — Modifier management

Staff Dashboard

- GET /dashboard/orders — Live order queue
- GET /dashboard/kitchen — Kitchen station view
- PATCH /dashboard/orders/:id/status — Advance workflow

- GET /dashboard/analytics — Branch / restaurant sales & top items

Platform Admin

- POST /admin/restaurants/:id/approve — Approve onboarding
- GET /admin/analytics — Platform metrics
- PATCH /admin/commissions — Commission settings
- GET /admin/audit-logs — Audit trail

9. QR Code Flow in Detail

Each table QR encodes an opaque, signed URL token rather than a raw table identifier. Example:
`https://app.tablebite.com/qr/8b2f1f...c9d?sig=...`

Resolution steps:

1. Verify cryptographic signature and token version.
2. Look up `qr_code` by `token_hash`.
3. Confirm `is_active` = true and (optional) `expires_at` not passed.
4. Confirm branch is open (or return closed-hours state).
5. If `table_id` present, attach table context to the short-lived guest session.
6. Return menu context; all subsequent orders inherit the server-resolved branch/table — client cannot override.

Anti-spoofing rules:

- Tokens are high-entropy and unguessable.
- Signature binds the payload; URL cannot be edited to switch tables.
- Orders always inherit server-resolved context.
- Reprinting a QR rotates `token_version` and invalidates previous links.
- Rate limits on the resolution endpoint prevent brute-force attempts.

10. Payment Flow in Detail

Card checkout uses Stripe in test mode during development and live mode in production. Environment variables keep the two worlds completely isolated. Cash / pay-at-counter is an explicit non-card path so the order workflow remains consistent.

Sequence:

1. Guest confirms cart and selects card or cash.
2. Backend calculates final amount from the server-side order snapshot (never trust client totals).
3. Card: NestJS creates Stripe PaymentIntent or Checkout Session.
4. Frontend confirms / redirects.
5. Stripe sends webhook to /payments/stripe/webhook.
6. Backend verifies signature, marks payment captured, advances order to placed/confirmed.
7. Socket.io notifies guest and staff.
8. Cash path: payment record stored as pending-due; staff marks settled later.

Environment isolation (example):

```
STRIPE_SECRET_KEY=sk_test_... # dev
STRIPE_PUBLISHABLE_KEY=pk_test_... # dev
STRIPE_WEBHOOK_SECRET=whsec_test_... # dev
STRIPE_SECRET_KEY=sk_live_... # prod
STRIPE_PUBLISHABLE_KEY=pk_live_... # prod
STRIPE_WEBHOOK_SECRET=whsec_live_... # prod
```

Never commit keys. Use different Stripe accounts or clearly separated test/live modes. Webhooks must be configured separately for test and live endpoints.

11. Folder / Project Structure

Monorepo (Turborepo or pnpm workspaces) keeps shared DTOs, generated Prisma types, UI primitives, and event contracts aligned between guest experience and staff dashboards.

```
monorepo/  
apps/  
web/ # Next.js App Router (customer + staff + admin UI)  
api/ # NestJS REST + Socket.io  
packages/  
ui/ # Shared design system (shadcn components, tokens)  
config/ # Shared TS / ESLint / Prettier  
types/ # Shared domain types & DTO helpers  
db/ # Prisma schema, migrations, seed  
realtime/ # Socket event contracts & helpers  
qr/ # Token signing & QR generation utilities  
storage/ # Upload helpers & media abstractions  
docs/  
planning/ # This PDF and future architecture notes  
.env.example  
turbo.json / pnpm-workspace.yaml
```

12. Non-Functional Requirements

12.1 Security

- Role-based authorization on every write endpoint and branch-scoped reads.
- Payment secrets never reach the browser; webhooks signature-verified.
- Rate limits on login, QR resolution, and checkout endpoints.
- Audit log for sensitive administrative changes.
- HTTPS everywhere; secure cookie / JWT settings; CSRF protection on state-changing routes.
- Input validation (class-validator / Zod) on all DTOs.

12.2 Performance

- Guest menu loads fast on mobile: aggressive image caching (CDN), SSR menu, lightweight JSON payloads.
- Order transitions near real-time via Socket.io; analytics can be asynchronous.
- Database indexes on foreign keys, status columns, and slug lookups.

12.3 Scalability

- Architecture scales from one restaurant to many without changing the public menu model.
- Stateless NestJS instances + Redis adapter for Socket.io; Postgres connection pooling.
- Horizontal scaling candidates: order creation, status events, QR session resolution.

12.4 Offline / Poor Connectivity

- Guest still sees last-loaded menu snapshot and clear retry state if connection drops.
- Staff dashboards degrade to polling fallback if sockets fail.
- Kitchen terminals keep recently loaded tickets visible until sync resumes.
- PWA manifest + service-worker readiness for offline menu caching in v2.

13. Implementation Roadmap

Ordered to get at least one restaurant serving real guests as early as possible. Payments and kitchen visibility arrive before deep platform-admin tooling because they directly affect order completion and operational reliability. Each phase ends with a git commit and a runnable demo checkpoint so work can be resumed cleanly if interrupted.

Phase	Scope (High Level)
0 — Bootstrap	Monorepo scaffolding, shared packages, Prisma init, CI, env example, basic health endpoints. First commit.
1 — Auth + Tenant + Menu + QR	Auth (email + Google), restaurant/branch setup, basic menu CRUD, public QR resolution, live menu view, availability toggles.
2 — Cart + Orders + Real-time	Cart, customization, order creation, status machine, Socket.io events, order history, customer accounts.
3 — Payments	Stripe PaymentIntent + webhook, card + cash fallback, receipts, environment isolation.
4 — Staff Ops	Order queue, kitchen view, table management, staff roles, branch analytics, browser print.
5 — Customer Polish	Favorites, reordering, ratings/reviews, search/filter polish, saved addresses.
6 — Platform Admin	Approval flows, commission config, basic payout tracking, platform analytics, audit logs.
7 — Hardening	Rate limits, observability, load testing, security review, PWA baseline, documentation for live key swap.

Recommendation: Start with Phase 0 + Phase 1. Lock auth, tenant model, menu schema, and the QR entry point before introducing cart, payments, or kitchen execution. After each phase the agent must commit with a descriptive message and ensure the application is in a runnable state.

14. Open Questions / Decisions (Resolved + Remaining)

14.1 Resolved Decisions (for this planning document)

Question	Decision	Rationale
Single-brand multi-branch vs multi-restaurant marketplace?	Multi-restaurant + multi-branch	Long-term product needs marketplace; branch model still supports chains cleanly.
Delivery in v1?	No — defer to v2	DSP integration, address validation, and SLA complexity delay core value.
Guest-only or full accounts at launch?	Both — guest checkout + optional accounts	Lowest friction for first order; accounts unlock history/favorites.
Cash handling?	Pay-at-counter (and COD later if delivery added)	Simple state model; staff settles later.
Recipe / inventory depletion in v1?	No — simple availability toggles only	Avoids stock model complexity until ordering is stable.
ORM?	Prisma	Type safety + migrations + monorepo alignment.
Real-time?	Socket.io + Redis adapter	Proven, scales horizontally, NestJS-friendly.
Component library?	Tailwind + shadcn/ui + Radix	Fast, accessible, consistent design system.

14.2 Remaining Decisions (can be answered during implementation or later)

- Exact commission default rate and whether it is platform-wide or per-restaurant editable from day one.
- Preferred email provider (Resend / Postmark / SES) and whether SMS is ever required in v1.
- Image storage preference (Cloudinary vs S3) and CDN strategy.
- Whether to support multiple currencies per platform or fix one currency per restaurant at launch.
- Exact kitchen station / multi-station routing model if a restaurant needs it early.

Appendix A — Sources & Research Links

Official / primary sources used for the competitive review (synthesized, not quoted at length):

- Toast Online Ordering — support.toasttab.com
- Square QR code ordering — squareup.com/us/en/online-ordering/qr-code-ordering
- GloriaFood — gloriafood.com
- Odoo Restaurant / Self-ordering — odoo.com documentation
- UpMenu — upmenu.com
- Open-source: github.com/elitale/restro, ERPNext Restaurant, URY, FloCafe, Satisfecho-related repos
- Stripe docs (keys, webhooks, PaymentIntents), NestJS auth & websockets, Prisma, Auth.js

The body of this document paraphrases and synthesizes the sources. Full URLs are retained in the original MesaFlow and TableBite planning PDFs for auditability.

Appendix B — Agent Execution Instructions

This planning document is the single source of truth for implementation. When the human gives the go-ahead:

1. Create the monorepo exactly as described in Section 11 (or the closest Turborepo/pnpm equivalent).
2. Begin with Phase 0 (bootstrap) and commit after a clean runnable skeleton exists.
3. Proceed phase by phase. After each phase:
 - Ensure the application builds and the relevant happy-path demo works.
 - Write a descriptive git commit (e.g. “feat(phase-1): auth, restaurant/branch, menu CRUD, public QR menu”).
 - Update a simple CHANGELOG or progress note so work can be resumed later.
4. Never hard-code Stripe keys or secrets; always use environment variables as specified.
5. Prefer the schema, API shapes, and security rules in this document. If a material change is required, document it in a short ADR under docs/.
6. When the human says “continue from the last commit”, inspect git log and resume the next unfinished phase.

Do not invent large new feature sets outside the roadmap without explicit approval. The goal is a solid, shippable core that can then grow.

*End of TableBite Ultimate Planning Document — Version 2.0
Ready for review and subsequent agent-driven implementation.*